



MSML610: Advanced Machine Learning

Lesson 09.2: Hidden Markov Models

Instructor: Dr. GP Saggese, gsaggese@umd.edu

References:

- *HMMs*
- Markov Random Fields
- Markov Logic Network
- Variational Inference

Algorithms for Specific Models

- **General temporal probabilistic reasoning** makes minimal assumptions:
 - Markov property for transitions
 - Markov property for sensor model
 - No constraints on:
 - * Mathematical form of transition/sensor models
 - * Nature of state and evidence variables (discrete or continuous)
- Improve efficiency and accuracy by exploiting specific model structures:
 - **Hidden Markov Models (HMMs)**:
 - * State is a single discrete variable
 - * Transition and observation models are discrete probability tables
 - * Enables fast algorithms like Viterbi, forward-backward, etc
 - **Kalman Filters**:
 - * State variables are continuous and normally distributed
 - * Linear Gaussian models for transitions and observations
 - * Allows exact, efficient updates using matrix operations
- Tailored algorithms can be orders of magnitude faster and more accurate than general methods

Hidden Markov Model: Formulation

- **Hidden Markov Model (HMM):** Temporal model with simplified structure for efficiency
 - **State model:**
 - * System state at time t is a discrete random variable $X_t \in \{1, \dots, S\}$
 - * E.g., in umbrella world, $X_t = \text{Rain}_t$ with states $\{\text{Rain}, \text{Sunny}\}$
 - * Can combine multiple variables into one “mega-state” variable
 - **Transition model** $\Pr(X_t|X_{t-1})$:
 - * Transition matrix $\underline{T}$ of size $S \times S$
 - * Entry $T_{ij} = \Pr(X_t = j|X_{t-1} = i)$: probability of transitioning from state i to j
 - **Sensor model:**
 - * Defined as $\Pr(E_t|X_t = i)$ for each state i
 - * Representable as a vector or diagonal matrix $\underline{O}$
 - * No assumptions about number or type (discrete/continuous) of observation variables
- **Benefit**
 - Enables efficient algorithms like forward, backward, and Viterbi

Hidden Markov Model: Example

- E.g., if $Rain = T$ is state 1 and $Rain = F$ is state 2, then the transition matrix for the umbrella world

R_{t-1}	$\Pr(R_t R_{t-1})$
T	0.7
F	0.3

becomes the transition model

$$\underline{T} = \begin{pmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{pmatrix}$$

- On day 1 we observe $U_1 = T$ and on day 3, $U_3 = F$, we have the observation matrices

$$\underline{O}_1 = \begin{pmatrix} 0.9 & 0 \\ 0 & 0.2 \end{pmatrix} \quad \underline{O}_3 = \begin{pmatrix} 0.1 & 0 \\ 0 & 0.8 \end{pmatrix}$$

Hidden Markov Model: Algorithms

- Using matrix representation all the forward / backward computations become matrix operations:

$$\mathbf{f}_{1:t+1} = \alpha \mathbf{O}_{t+1} \mathbf{T}^\top \mathbf{f}_{1:t}$$
$$\mathbf{b}_{k+1:t} = \mathbf{T} \mathbf{O}_{k+1} \mathbf{b}_{k+2:t}$$

- Express inference tasks (e.g., filtering, smoothing) as efficient matrix multiplication
- Specialized algorithms to improve time and space complexity

Hidden Markov Model: Algorithms

- Baum-Welch
 - Special case of Expectation-Maximization (EM) algorithm
 - Pros: Converges to local maximum of likelihood
 - Cons: Only point-estimation, no uncertainty estimation
- Viterbi
 - Finds most likely sequence of hidden states
 - Pros: Fast approximation of BW
 - Cons: Returns local optimum
- Gradient-based methods
 - Use gradient descent to optimize parameters
 - Pros: Fast
 - Cons: Needs differentiable model
 - E.g., PyTorch / TensorFlow probability
- HMM with MCMC
 - Learn posterior distribution of parameters using Bayesian inference
 - Pros: Flexible, accounts for uncertainty
 - Cons: Computationally expensive
 - E.g., PyMC

Hidden Markov Model: Applications

- HMMs model systems with hidden states producing observable outputs
- **Audio / speech**
 - Speech recognition: map audio to phonemes, words
 - Speaker identification: model vocal traits to recognize a speaker
 - Music generation and transcription
- **Biology / genomics**
 - Gene prediction: find DNA regions
 - Protein structure prediction
- **Finance / economics**
 - Market regime detection: bull/bear markets, volatility regimes
 - Credit scoring: observe purchases, estimate financial health (hidden variable)

Hidden Markov Model: Applications

- **Security / anomaly detection**
 - User behavior modeling: detect anomalous login patterns or usage
 - Intrusion detection: model normal traffic to spot attacks
 - Fraud detection: identify unusual transactions
- **NLP**
 - Part-of-speech tagging: map words to syntactic roles
 - Named entity recognition: identify entities, people, places
- **Operations and process monitoring**
 - Predictive maintenance: model machine health from sensor readings
 - Process monitoring: detect deviation from normal operations
 - Customer behavior modeling: understand customer intent
- **Environmental monitoring**
 - Weather prediction: infer atmospheric state from observed variables

Hidden Markov Model: Limitations

- **Short memory**
 - Markov assumption: current state depends only on previous state
 - Inefficient for long-range dependencies
- **Predefined, fixed number of states**
 - Mis-estimating states leads to underfitting or overfitting
- **Stationarity assumption**
 - Transition and sensor probabilities constant over time
- **Use atomic representation**
 - States are labels with no internal structure
 - Hard to interpret with many states or unclear state meanings
- **Training** is computationally expensive for large datasets
 - Struggles with sparse data
- **Alternatives**
 - Bayesian networks using factored representation
 - Deep learning handles complex temporal dependencies and long-term relationships

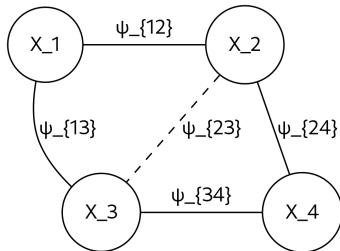
- HMMs
- *Markov Random Fields*
- Markov Logic Network
- Variational Inference

Markov Random Fields

- A **Markov Random Field** is a probabilistic graphical model
 - Represents a joint distribution using an undirected graph
 - * Nodes = random variables
 - * Edges = relationships (dependencies) between variables
 - Key idea: **Markov property**
 - * Each variable is conditionally independent of non-neighbors given its neighbors
 - Model **spatial and contextual dependencies**
 - * Capture local interactions that combine into a global structure
- **Example:** Image de-noising
 - Each pixel tends to have similar intensity to its neighbors
 - Noise introduces local inconsistencies
 - MRF models smoothness while respecting observed data
- **Example:** Social networks
 - Friends influence each other's behavior
 - Dependencies exist only among connected individuals

Markov Random Fields: Model Form

- $Pr(X) = \frac{1}{Z} \prod_{C \in \mathcal{C}} \psi_C(X_C)$
- X : set of all random variables in the model
- $\mathcal{C}$: set of cliques in the graph (fully connected subset of nodes)
- $\psi_C(X_C)$: **potential function** for clique C
 - Assigns a positive score to each possible configuration of variables in C
 - Clique potentials ψ_C encode preferences or constraints
 - Intuition: measures “compatibility” of values
 - High ψ_C = compatible configuration
 - Low ψ_C = unlikely configuration
- Z : **partition function**
 - Ensures probabilities sum to 1
 - Usually very hard to compute for



- HMMs
- Markov Random Fields
- *Markov Logic Network*
- Variational Inference

Markov Logic Networks: Intuition

- **Intuition:**
 - Logic rules are often *soft* (have exceptions) rather than *absolute* (true or false)
- Example: Social network friendships
 - Rule: “Friends of friends are likely friends”
 - Not always true, but often holds
- Example: Natural language processing
 - Rule: “Every sentence has a subject” weighted by importance
- Markov Logic Networks
 - Unify knowledge representation (logic) with uncertainty handling (probability)
 - Allow violations of rules but penalize them probabilistically
- Applications:
 - Information extraction
 - Entity resolution
 - Relational learning
- Main challenge: inference and learning are computationally expensive

Markov Logic Networks: Basics

- A **Markov Logic Network** (MLN) combines:
 - First-order logic (expressing knowledge with rules and quantifiers)
 - Markov Random Fields (modeling uncertainty with probabilities)
- Each element is a pair (F_i, w_i)
 - F_i : a first-order logic formula
 - * E.g., " $Friends(x, y) \implies Similar(x, y)$ "
 - w_i : a weight measuring the strength of belief in F_i
 - * Higher w_i = formula more important in shaping the probability distribution
- **Semantics:**
 - An MLN defines a probability distribution over *possible worlds*
 - A world = a complete assignment of truth values to all ground atoms
 - If a world satisfies many high-weight formulas, it becomes *more probable*
- **Joint distribution:**
 - $\Pr(X = x) = \frac{1}{Z} \exp(\sum_i w_i n_i(x))$
 - $n_i(x)$ = number of **true groundings** of formula F_i in world x
 - * Example: If $F_i = "Friends(x, y) \rightarrow Similar(x, y)"$ and in world x this holds for 7 pairs (x, y) out of 10, then $n_i(x) = 7$
- **Special cases:**

- HMMs
- Markov Random Fields
- Markov Logic Network
- **Variational Inference**
 - Expectation-Maximization (EM) Algorithm

- HMMs
- Markov Random Fields
- Markov Logic Network
- Variational Inference
 - *Expectation-Maximization (EM) Algorithm*

EM Algorithm: Intuition and Applications

- Expectation-Maximization (EM) is a method for learning with hidden or missing data
 - Useful when some variables influencing the data are not directly observed
 - Works by iteratively improving parameter estimates
 - Alternates between estimating missing data and optimizing parameters
- Two main steps:
 - **E-step (Expectation)**: Estimate distribution over hidden variables using current parameters
 - **M-step (Maximization)**: Update parameters to maximize expected log-likelihood from the E-step
- Used in diverse settings:
 - Unsupervised clustering (e.g., Gaussian Mixture Models)
 - Learning with incomplete data in Bayesian networks
 - Hidden Markov Models (HMMs)
- Key property: EM increases data likelihood at each iteration
- Converges to a local maximum of the likelihood function
- No need for a step size parameter unlike gradient descent

EM Algorithm: Mechanics and Example in Gaussian Mixture Models

- Goal: Recover parameters of Gaussian components from unlabeled data
- **E-step:**
 - Compute $p_{ij} = P(C = i \mid x_j)$ using Bayes' rule
 - $p_{ij} \propto P(x_j \mid C = i)P(C = i)$
 - Calculate effective count: $n_i = \sum_j p_{ij}$
- **M-step:**
 - Update means: $\mu_i \leftarrow \sum_j p_{ij}x_j / n_i$
 - Update covariances: $\Sigma_i \leftarrow \sum_j p_{ij}(x_j - \mu_i)(x_j - \mu_i)^T / n_i$
 - Update weights: $w_i \leftarrow n_i / N$
- Intuition: Softly assign points to components, then re-estimate the components
- Example scenario:
 - 500 data points from a mix of 3 Gaussians
 - EM reconstructs original distribution closely after iterations
- Limitations:
 - Sensitive to initialization
 - May converge to poor local optima
 - Component collapse or merging can occur

Introduction to the Expectation–Maximization (EM) Algorithm

- **Purpose of EM Algorithm**

- Iterative method for finding maximum likelihood or maximum a posteriori (MAP) estimates in statistical models with latent variables
- Particularly useful when data is incomplete or has missing values

- **Key Concepts**

- **Observed Data ($\mathbf{X}$)**: The data we can directly observe
- **Latent Variables ($\mathbf{Z}$)**: Hidden or unobserved variables that influence the observed data
- **Parameters (θ)**: Unknown parameters to be estimated

- **Challenge Addressed**

- Direct maximization of the likelihood function $p(\mathbf{X}|\theta)$ is often intractable due to the presence of latent variables

- **EM Algorithm Overview**

- Alternates between estimating the expected value of the log-likelihood (E-step) and maximizing this expectation (M-step)

- **Applications**

- Widely used in clustering (e.g., Gaussian Mixture Models), natural language processing, and image reconstruction

The EM Algorithm: Step-By-Step

- **Initialization**

- Start with initial guesses for the parameters $\theta^{(0)}$

- **E-Step (Expectation Step)**

- Compute the expected value of the log-likelihood function, with respect to the conditional distribution of the latent variables given the observed data and current parameter estimates:

- * $Q(\theta|\theta^{(t)}) = \mathbb{E}_{\mathbf{Z}|\mathbf{X},\theta^{(t)}}[\log p(\mathbf{X}, \mathbf{Z}|\theta)]$

- **M-Step (Maximization Step)**

- Maximize the expected log-likelihood found in the E-step to update the parameters:

- * $\theta^{(t+1)} = \arg \max_{\theta} Q(\theta|\theta^{(t)})$

- **Iteration**

- Repeat E and M steps until convergence, i.e., until the parameters stabilize or the increase in likelihood is below a threshold

Mathematical Foundation of EM

- **Likelihood with Latent Variables**

- The marginal likelihood of the observed data is:

- * $p(\mathbf{X}|\theta) = \int p(\mathbf{X}, \mathbf{Z}|\theta) d\mathbf{Z}$

- **Intractability**

- The integral is often difficult to compute due to the complexity introduced by the latent variables

- **EM Solution**

- EM circumvents this by iteratively applying the E and M steps to find parameter estimates that locally maximize the likelihood

- **Convergence**

- Each iteration of EM is guaranteed to increase the likelihood function, ensuring convergence to a local maximum

Example: Gaussian Mixture Models (GMM)

- **Problem Setup**
 - Data is assumed to be generated from a mixture of Gaussian distributions, each with its own mean and covariance
- **Latent Variables**
 - Each data point is associated with a latent variable indicating the Gaussian component from which it was generated
- **E-Step in GMM**
 - Compute the posterior probabilities (responsibilities) that each data point belongs to each Gaussian component
- **M-Step in GMM**
 - Update the parameters (means, covariances, and mixing coefficients) of each Gaussian component using the responsibilities computed in the E-step
- **Iteration**
 - Repeat E and M steps until the parameters converge

Properties and Limitations of EM

- **Advantages**

- Can handle missing or incomplete data effectively
- Provides a framework for parameter estimation in complex models

- **Limitations**

- Converges to a local maximum, which may not be the global maximum
- Sensitive to initial parameter estimates; poor initialization can lead to suboptimal solutions

- **Extensions and Variants**

- **Variational Bayes**: Provides a fully Bayesian approach by estimating distributions over parameters
- **Generalized EM (GEM)**: Relaxes the requirement of fully maximizing the expected log-likelihood in the M-step
- **Expectation Conditional Maximization (ECM)**: Breaks the M-step into several conditional maximization steps

- **Practical Considerations**

- Multiple runs with different initializations can help in finding better solutions
- Monitoring the increase in likelihood can help in determining convergence